

An Introduction to Automated Software Testing using Symbolic Execution

nmeum

2026-07-17

nmeum@hannover.ccc.de

Motivation



Goal: Ensuring that newly written code behaves as expected
↳ By comparing generated and expected *output* for selected *inputs*

Goal: Ensuring that newly written code behaves as expected

↳ By comparing generated and expected *output* for selected *inputs*

- Example: `assert(myfunc(input1, input2) == expected_output);`

Goal: Ensuring that newly written code behaves as expected

↳ By comparing generated and expected *output* for selected *inputs*

- Example: `assert(myfunc(input1, input2) == expected_output);`
- Requires selection of appropriate inputs → writing tests is a *manual process*
- Significant amount of time during software engineering is spent on testing

Have I tested enough?

Problem: Determining when software has been sufficiently tested is challenging
↳ When can I stop adding more test cases to my test suite?

Have I tested enough?

Problem: Determining when software has been sufficiently tested is challenging
↳ When can I stop adding more test cases to my test suite?

- Quality of test cases is approximated through coverage metrics
- Coverage metrics differ with respect to the coverage target
- Examples include line, branch, or statement coverage

Limitations of Branch Coverage

```
1  int myfunc(bool a, bool b) {  
2      int x = 23, y = 42;  
3  
4      if (a)  
5          x = 0;  
6      else  
7          y = 0;  
8  
9      if (b)  
10         return 5 / x;  
11     else  
12         return 5 / y;  
13 }
```

Complete branch coverage with two inputs:

Limitations of Branch Coverage

```
1  int myfunc(bool a, bool b) {  
2      int x = 23, y = 42;  
3  
4      if (a)  
5          x = 0;  
6      else  
7          y = 0;  
8  
9      if (b)  
10         return 5 / x;  
11     else  
12         return 5 / y;  
13 }
```

Complete branch coverage with two inputs:

1. $a \leftarrow \text{True}, b \leftarrow \text{False}$

Limitations of Branch Coverage

```
1  int myfunc(bool a, bool b) {  
2      int x = 23, y = 42;  
3  
4      if (a)  
5          x = 0;  
6      else  
7          y = 0;  
8  
9      if (b)  
10         return 5 / x;  
11     else  
12         return 5 / y;  
13 }
```

Complete branch coverage with two inputs:

1. $a \leftarrow \text{True}, b \leftarrow \text{False}$
2. $a \leftarrow \text{False}, b \leftarrow \text{True}$

Limitations of Branch Coverage

```
1  int myfunc(bool a, bool b) {  
2      int x = 23, y = 42;  
3  
4      if (a)  
5          x = 0;  
6      else  
7          y = 0;  
8  
9      if (b)  
10         return 5 / x;  
11     else  
12         return 5 / y;  
13 }
```

Complete branch coverage with two inputs:

1. $a \leftarrow \text{True}, b \leftarrow \text{False}$
2. $a \leftarrow \text{False}, b \leftarrow \text{True}$

Limitations of Branch Coverage

```
1  int myfunc(bool a, bool b) {  
2      int x = 23, y = 42;  
3  
4      if (a)  
5          x = 0;  
6      else  
7          y = 0;  
8  
9      if (b)  
10         return 5 / x;  
11     else  
12         return 5 / y;  
13 }
```

Complete branch coverage with two inputs:

1. $a \leftarrow \text{True}, b \leftarrow \text{False}$
2. $a \leftarrow \text{False}, b \leftarrow \text{True}$

But, these test cases miss a div-by-zero bug:

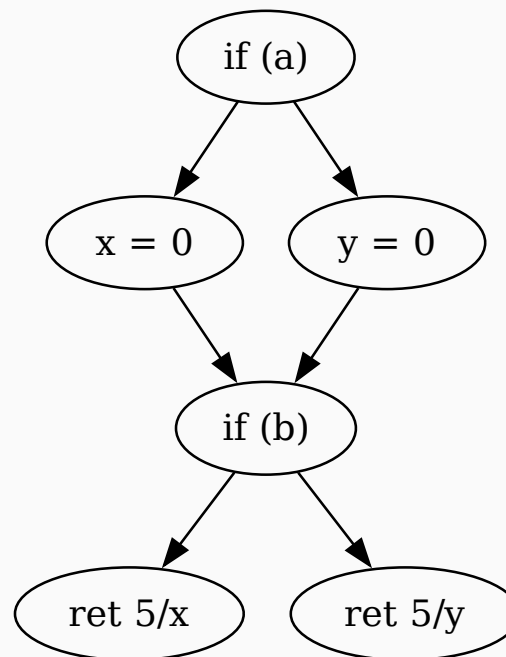
- Occurs when a and b have the same value
- Problem: Each branch is viewed in isolation

Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: []

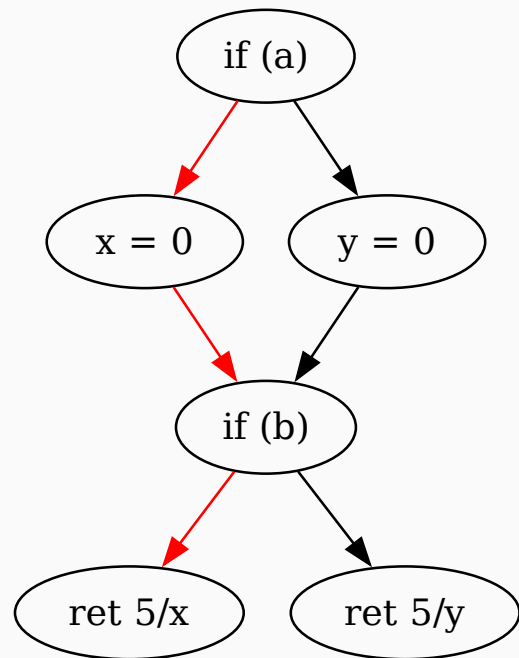


Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: [(T, T)

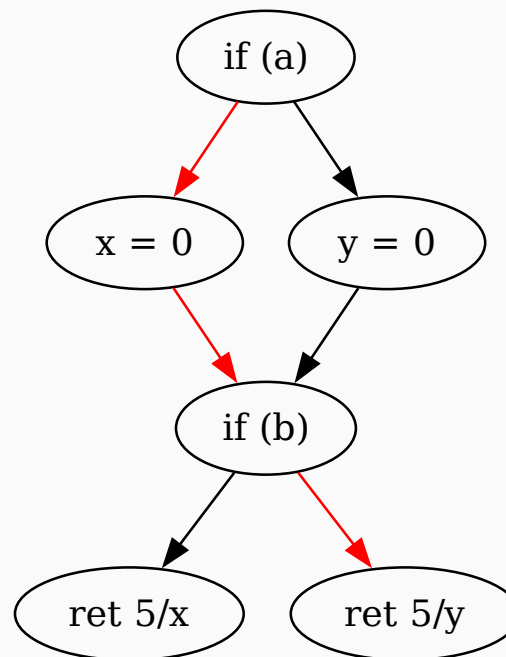


Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: [(T, T), (T, F)]

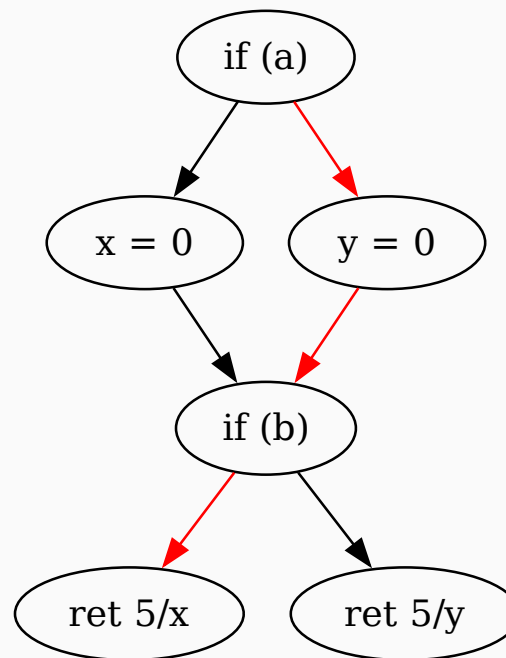


Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: [(T, T), (T, F), (F, T)]

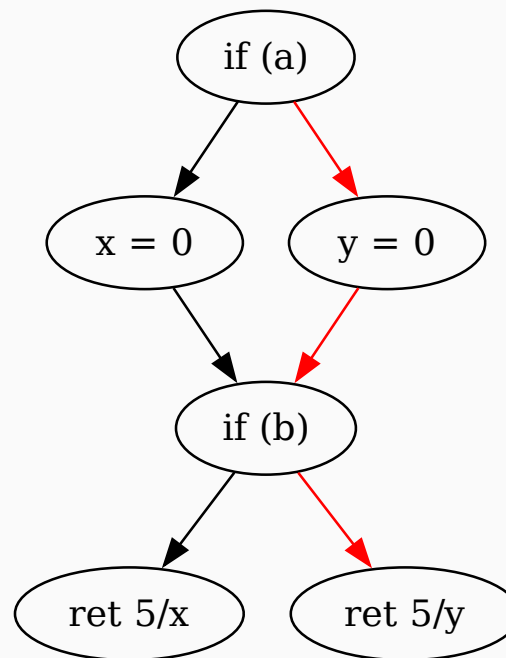


Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: [(T, T), (T, F), (F, T), (F, F)]
- We need 4 test cases to cover all execution paths

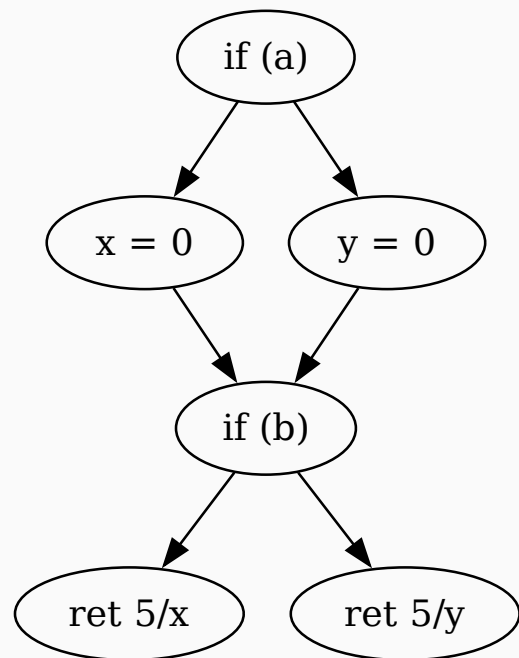


Covering Paths, not Branches

Solution: Cover *execution paths* instead of *branches*

↳ Thus including the order in which branches are taken

- Path specifies taken edges, for every branch point
- Best visualized through a control-flow graph (CFG)
- For myfunc: [(T, T), (T, F), (F, T), (F, F)]
- We need 4 test cases to cover all execution paths



Paths grow *exponentially* with branches in the code

↳ **Path coverage:** How many paths have we covered?

Challenges:

1. Writing test cases manually is impractical for abundant path coverage
2. Some execution paths may be infeasible due to unsatisfiable conditions

Challenges wrt. Path Coverage

Challenges:

1. Writing test cases manually is impractical for abundant path coverage
2. Some execution paths may be infeasible due to unsatisfiable conditions

Example:

```
if (a > 8)
...
if (a < 5)
...
```

- a cannot be > 8 and < 5 at the same time
- Path where both are taken is *not feasible!*
- Cannot come up with a test input for this path

Automating Test Input Generation

Generating Inputs for Path Coverage

Goal: Automate input generation with path coverage in mind

- Requires reasoning about satisfiability of branch conditions
- Branch conditions are logical formulas, such as $(a \ \&\& \ b) \ || \ c$
- Need to find a satisfying assignment for variables in these formulas
 - Which values are required for a , b , and c so that branch is taken?
 - If no such assignment exists the branch is not *feasible*

- This is the **Boolean satisfiability problem** (SAT) in disguise
 - ▶ Input: Logical formula f in propositional logic
 - ▶ Output:
 - Variable assignment for f so that it evaluates to true
 - Or: Proof that no such assignment exists

Example: $f := (x_1 \text{ or } x_2) \text{ and } (x_1 \text{ or } !x_2) \text{ and } (!x_1 \text{ or } x_2)$

$\Rightarrow$ Satisfying assignment: $x_1 = \text{True}, x_2 = \text{True}$

- This is the **Boolean satisfiability problem** (SAT) in disguise
 - Input: Logical formula f in propositional logic
 - Output:
 - Variable assignment for f so that it evaluates to true
 - Or: Proof that no such assignment exists

Example: $f := (x_1 \text{ or } x_2) \text{ and } (x_1 \text{ or } !x_2) \text{ and } (!x_1 \text{ or } x_2)$

$\Rightarrow$ Satisfying assignment: $x_1 = \text{True}, x_2 = \text{True}$

- This is the **Boolean satisfiability problem** (SAT) in disguise
 - Input: Logical formula f in propositional logic
 - Output:
 - Variable assignment for f so that it evaluates to true
 - Or: Proof that no such assignment exists
- Known SAT solving algorithms have exponential worst-case complexity
- But: Modern SAT solvers can handle formulas with thousands of variables

Satisfiability Modulo Theories

Problem: SAT is defined over Boolean values and logic operations

↳ For reasoning about real-world code we also need arithmetic operations

- An extension of SAT: Satisfiability modulo theories (SMT)
- Supports more high-level theories (BitVectors!)
- Standard to express SMT formulas: SMT-LIB
 - ▶ The input format processed by SMT solvers (e.g., Z3)

```
(set-logic QF_BV)
(declare-const x (_ BitVec 32))
(assert
  (= (bvurem (bvmul x (_ bv2 32))
             (_ bv2 32))
     (_ bv0 32)))
(check-sat)
```

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

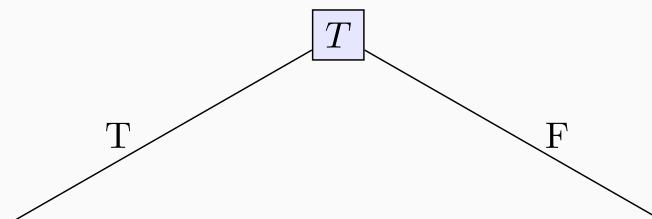
- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2     if (n < 0)  
3         n = n * -1;  
4     else  
5         n = 1;  
6  
7     if (n <= 0)  
8         abort();  
9 }
```

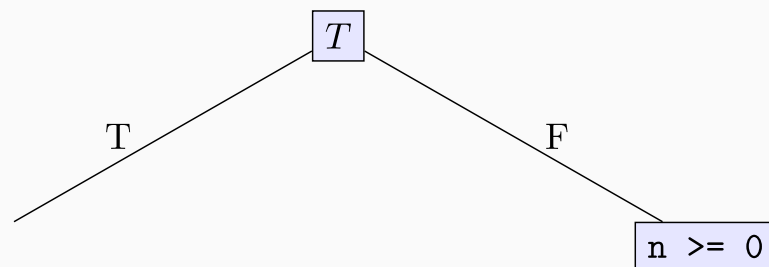


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```

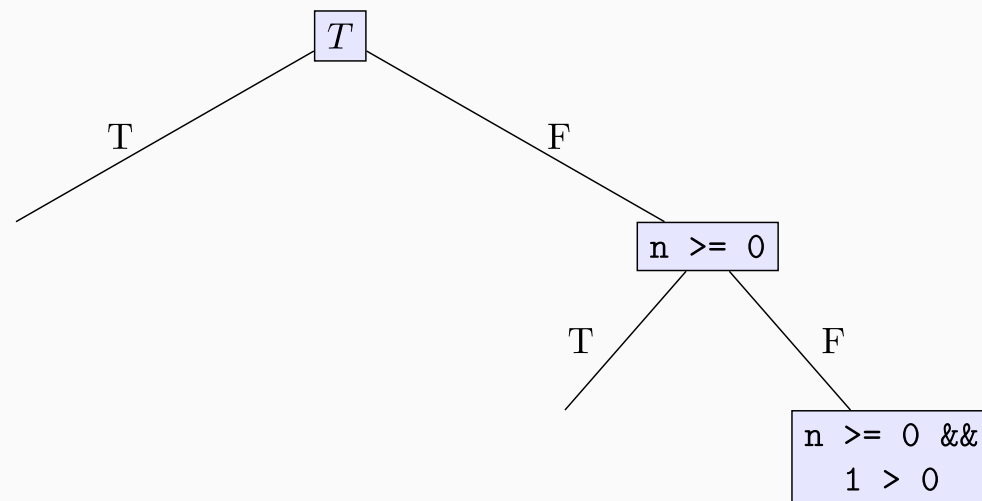


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```

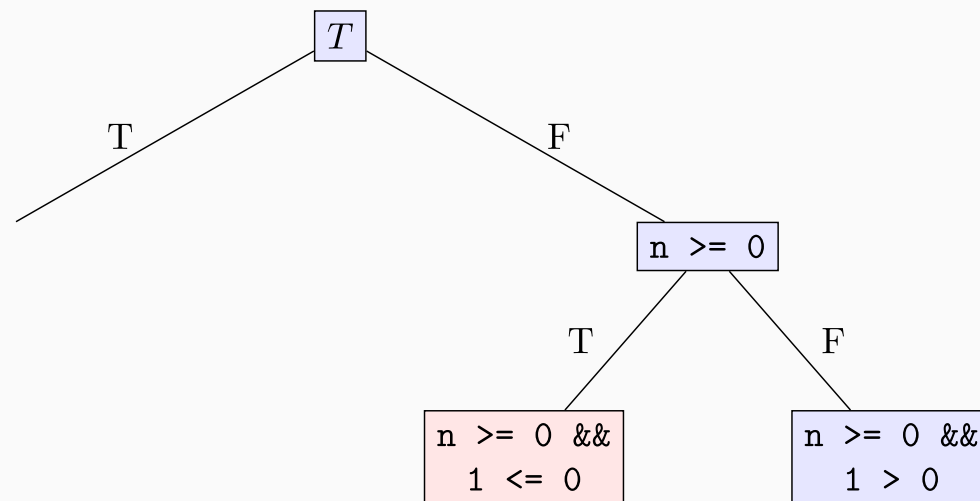


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```

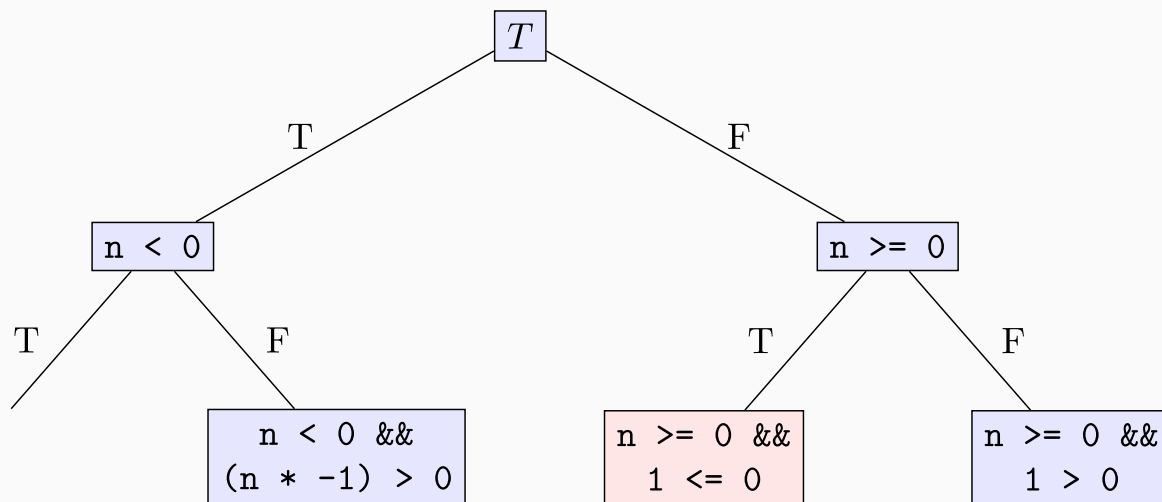


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```

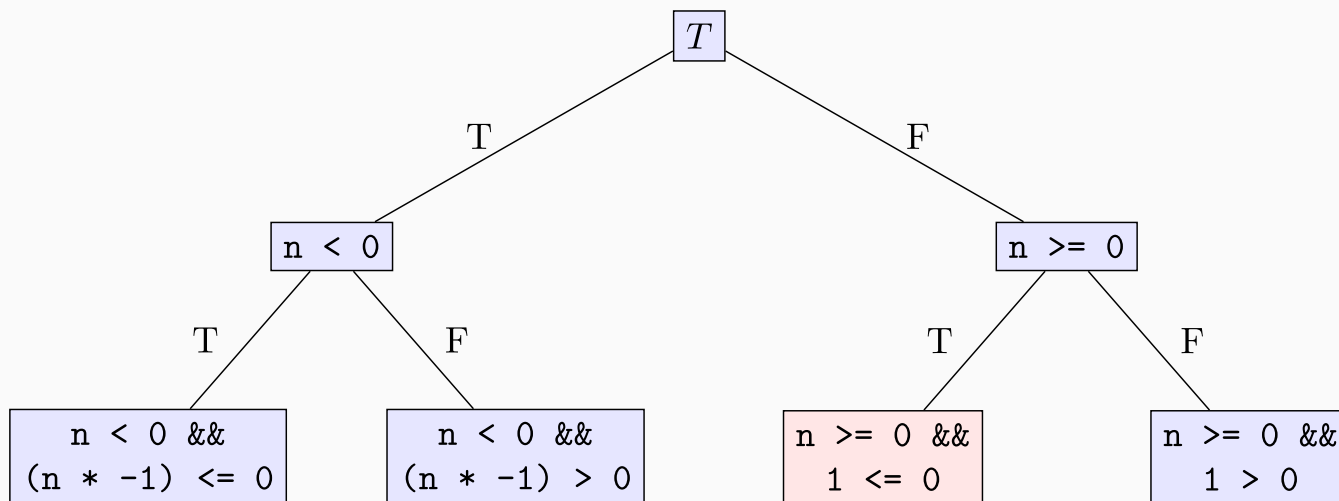


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```

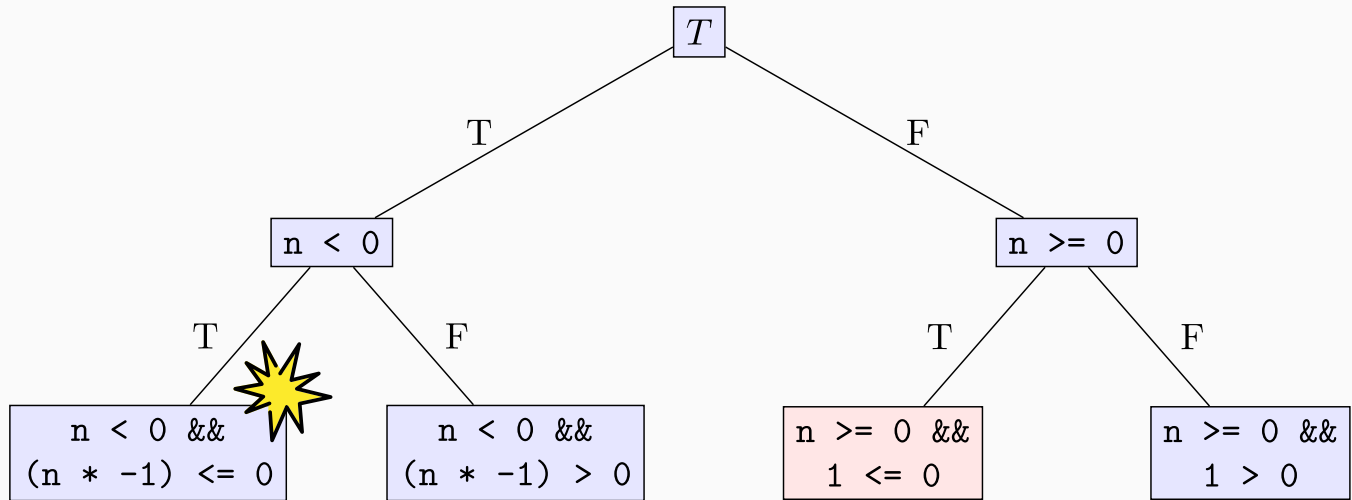


Symbolic Execution

Symbolic Execution: Treats selected program values as symbolic SMT-LIB variables

- Symbolic variables refer to a set of possible concrete values (e.g., $x > 5$)
- These variables are *continuously constrained* along the execution path

```
1 void somefunc(int n) {  
2   if (n < 0)  
3     n = n * -1;  
4   else  
5     n = 1;  
6  
7   if (n <= 0)  
8     abort();  
9 }
```



Transforming Program Semantics to SMT-LIB

Needed: Transformation of branch conditions in tested code to SMT-LIB formulas

- Modern languages have very complicated semantics (cumbersome transformation)
- Ideally, we also want to employ symbolic execution for different source languages

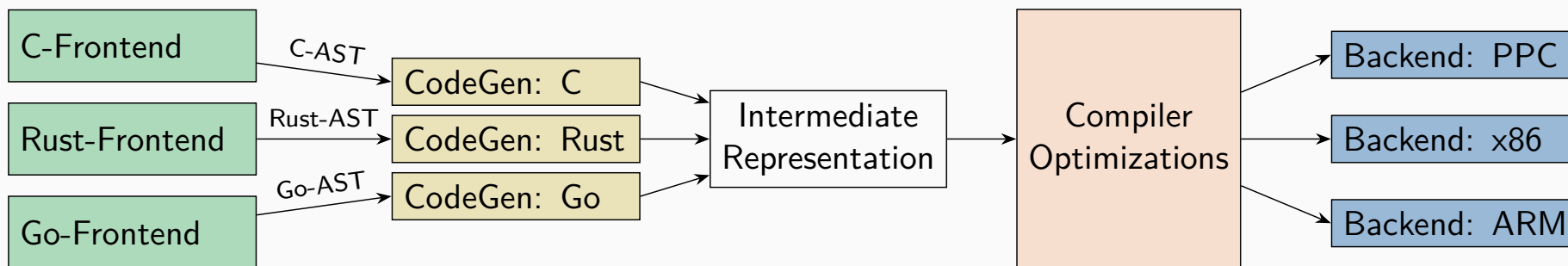
Transforming Program Semantics to SMT-LIB

Needed: Transformation of branch conditions in tested code to SMT-LIB formulas

- Modern languages have very complicated semantics (cumbersome transformation)
- Ideally, we also want to employ symbolic execution for different source languages

Solution: Operate on an *intermediate representation* of the program's semantics

↳ Simpler specification, applicability to different programming languages



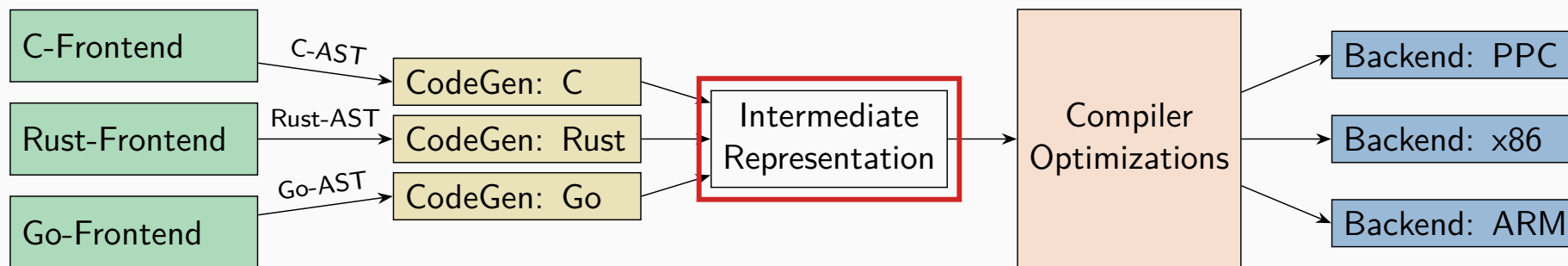
Transforming Program Semantics to SMT-LIB

Needed: Transformation of branch conditions in tested code to SMT-LIB formulas

- Modern languages have very complicated semantics (cumbersome transformation)
- Ideally, we also want to employ symbolic execution for different source languages

Solution: Operate on an *intermediate representation* of the program's semantics

↳ Simpler specification, applicability to different programming languages



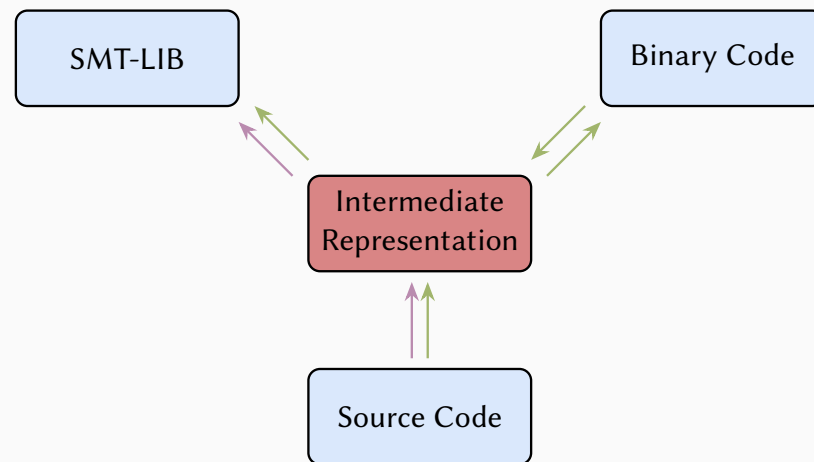
Example: LLVM IR

- LLVM IR is the IR of the LLVM compiler infrastructure
 - Emitted by Clang for C but also by the Rust compiler
 - Essentially a portable assembly language
- Symbolic execution generate SMT-LIB from such IRs
 - Providing a mapping: IR operation $\mapsto$ SMT-LIB

```
define i32 @fac(i32 %0) {  
    br label %2  
2:  
    %3 = phi i32 [ 1, %1 ], [ %8, %6 ]  
    %4 = phi i32 [ %0, %1 ], [ %7, %6 ]  
    %5 = icmp eq i32 %4, 0  
    br i1 %5, label %9, label %6  
6:  
    %7 = add nsw i32 %4, -1  
    %8 = mul nsw i32 %3, %4  
    br label %2  
9:  
    %10 = mul nsw i32 %3, 1  
    ret i32 %10  
}
```

Binary- vs. Source-based Symbolic Execution

- IRs originate from the compiler construction area
 - LLVM IR
 - GCC GIMPLE
- Also possible to generate the IR for binary code
 - Through a process referred to as *binary lifting*
 - Allows symbolic execution of binary code



From Theory to Praxis



Existing Implementations

KLEE: Symbolic execution engine for LLVM



- Source-based symbolic execution
- Symbolically executes *LLVM IR*
- Monolithic C++ code base

angr: Python-based binary analysis platform



- Binary-based symbolic execution
- Symbolically executes *Valgrind Vex*
- Modular Python library

Existing Implementations

KLEE: Symbolic execution engine for LLVM



- Source-based symbolic execution
- Symbolically executes *LLVM IR*
- Monolithic C++ code base

angr: Python-based binary analysis platform



- Binary-based symbolic execution
- Symbolically executes *Valgrind Vex*
- Modular Python library

Alternative Implementations:

- **BinSec:** OCaml framework for symbolic binary analysis
- **Owi:** Symbolic execution for WebAssembly binaries
- **CrossHair:** Symbolic testing framework for Python

- Base64: An ASCII encoding technique for binary data
- Want to verify a Base64 implementation written in C

- Base64: An ASCII encoding technique for binary data
- Want to verify a Base64 implementation written in C

Desired property:

```
base64_decode(base64_encode(data)) == data
```

- Base64: An ASCII encoding technique for binary data
- Want to verify a Base64 implementation written in C

Desired property:

`base64_decode(base64_encode(data)) == data`

Goal: Show that this property holds using KLEE

Live Demo

Scalability: In our example we enumerate all paths — the exploration is complete!

↳ Sadly, for more complex programs this is not always feasible in practice

1. *Path explosion:* Exponential growth, full enumeration quickly becomes challenging
2. *Solver time:* Solving constraints becomes increasingly more time-consuming

Scalability: In our example we enumerate all paths — the exploration is complete!

↳ Sadly, for more complex programs this is not always feasible in practice

1. *Path explosion:* Exponential growth, full enumeration quickly becomes challenging
2. *Solver time:* Solving constraints becomes increasingly more time-consuming

However: Works surprisingly well for medium-complexity programs

↳ Also: Divide-and-conquer, varying input size, combinations with fuzzing, ...

Application to GNU Coreutils

- KLEE was **originally evaluated** on coreutils and BusyBox
- Test cases for coreutils were automatically generated
- Coverage comparison with existing manually-written tests

Coverage (w/o lib)	COREUTILS		BUSYBOX	
	KLEE tests	Devel. tests	KLEE tests	Devel. tests
100%	16	1	31	4
90-100%	40	6	24	3
80-90%	21	20	10	15
70-80%	7	23	5	6
60-70%	5	15	2	7
50-60%	-	10	-	4
40-50%	-	6	-	-
30-40%	-	3	-	2
20-30%	-	1	-	1
10-20%	-	3	-	-
0-10%	-	1	-	30
Overall cov.	84.5%	67.7%	90.5%	44.8%
Med cov/App	94.7%	72.5%	97.5%	58.9%
Ave cov/App	90.9%	68.4%	93.5%	43.7%

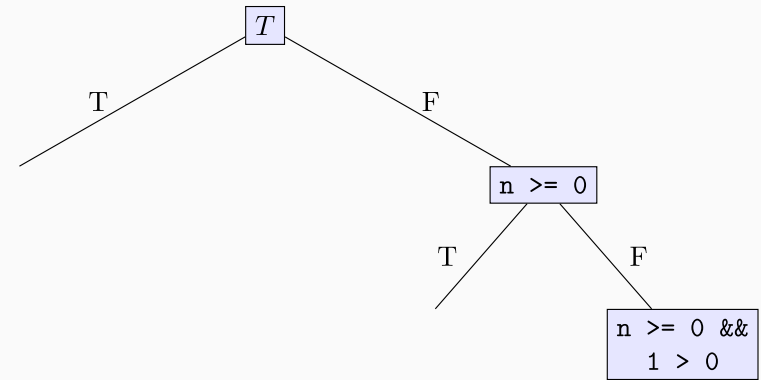
Table created by Cadar et al., taken from the 2008 OSDI publication on KLEE.

Outlook



Symbolic Execution for Bug Finding

- **Bug finding:** Partial exploration of the program
- Cover only a subset of all reachable execution paths
 - Similar to Fuzzing (we are only looking for bugs)
 - Prioritize exploration of “buggy paths”
- *Path Selection:* Which path to explore next?



- Apart from exploring paths, we also need to check them for errors
- So far: We were primarily interested in failing asserts
- However, KLEE can also check for non-functional bugs:
 - ▶ Undefined behavior through UBSAN
 - ▶ Memory safety violations
 - ▶ ...

Takeaways:

1. Symbolic execution can automate test input generation for us
2. Contrary to fuzzing, it takes the program structure into account
3. Scalability is a challenge, partial exploration is the common case

Takeaways:

1. Symbolic execution can automate test input generation for us
2. Contrary to fuzzing, it takes the program structure into account
3. Scalability is a challenge, partial exploration is the common case

🌟 Give it a try, find and fix some bugs! 🌟

- Try the Base64 verification example at home
- Play around with the various KLEE tutorials

⇒ See <https://git.8pit.net/symex-intro>